

Requerimientos integrales de infraestructura de BI

Fase inicial: tableros y análisis orientados al **seguimiento de la cobranza** sobre la cartera de las líneas que hoy comercializa La Fe (**funerario, vida AP y RCV**). **Etapas posteriores:** se espera ampliar el BI a **otras áreas** de forma escalonada; la infraestructura debe estar planteada para ese crecimiento. Este documento explica, en lenguaje claro, qué se necesita de tecnología y por qué, sin poner en riesgo el sistema operativo diario.

Fecha 14/04/2026 · **Versión** 1.1 · **Estado** Propuesta para validación por sistemas

Seguros La Fe · Más de 60 años de trayectoria · Una empresa de la Organización Vallés

Elaborado por: Angel Colmenares

Cargo: Coordinador Estadístico

Nota: este documento resume, en lenguaje accesible, qué debe facilitar el área de tecnología para desplegar el proyecto de BI en cobranza de forma segura y sostenible; el autor no es especialista en administración de servidores. Los requerimientos son una **propuesta base**, abiertos a **ajuste y mejora** con los aportes de los **especialistas de cada ámbito** de la compañía (tecnología, seguridad, negocio, cumplimiento, etc.), sin sustituir el criterio ni la responsabilidad formal de cada dominio.

Documento único

Uso interno

Lectura mixta

(equipos técnicos y de negocio)

Qué se solicita y justificación (en breve)

La compañía dispone de un **core de seguros (Sirweb)** y de su **base de datos**, donde vive la información operativa diaria. Hoy la oferta se concentra en **funerario, vida AP y RCV**. En esta primera etapa el área de cobranza necesita **indicadores y reportes centrados en el seguimiento de la cobranza** (mora, vencimientos, cartera y demás métricas que defina negocio sobre esas líneas) en formato de **inteligencia de negocio (BI)**: gráficos, tablas y filtros que apoyen el día a día, sin depender solo de extractos manuales.

Para lograrlo sin sobrecargar ni poner en riesgo el sistema transaccional, se requiere un **entorno separado** donde puedan ejecutarse las aplicaciones (pantallas en **Streamlit**, más una **API en Python** que concentre las reglas y los números) y donde exista un **lugar autorizado para guardar tablas intermedias** (resultados de cálculos, agregados) que no deben escribirse sobre la base de solo lectura que suelen dar las réplicas.

La **justificación** es triple: **(1) continuidad operativa** — el core sigue atendiendo la operación aseguradora del día a día sin ralentizarse por consultas pesadas de analítica; **(2) seguridad y gobierno del dato** — accesos acotados, HTTPS, red interna o VPN, y políticas de acceso acordadas (p. ej. SSO en fases posteriores); **(3) evolución por fases** — el mismo esquema técnico puede incorporar **más KPIs** y, más adelante, **BI en otras áreas** de la compañía de forma escalonada, evitando fragmentar la plataforma.

Lo que sigue detalla **qué infraestructura concreta** se pide (servidor, Docker, bases de datos, red). Al **final del documento** se incluye un **anexo con glosario** para quienes no trabajan a diario con estos términos.

1. Objetivo

Disponer de un entorno **estable, seguro y escalable** para:

- Tableros y análisis (**Streamlit** u homólogo acordado), con **énfasis inicial en cobranza** y posibilidad de ampliar a otras áreas según roadmap.
- **API en Python** (p. ej. FastAPI) como capa de métricas y reglas de negocio frente a los datos.
- **No impactar** el sistema transaccional del core (Sirweb): la lectura operativa de datos sensibles se hará por canales definidos por tecnología.
- **Infraestructura dimensionada** para crecer con el volumen de datos y la **conurrencia de acceso**, y para **incorporar en etapas posteriores** nuevos tableros o fuentes alineadas a otras áreas, sin obligar a rediseñar desde cero.

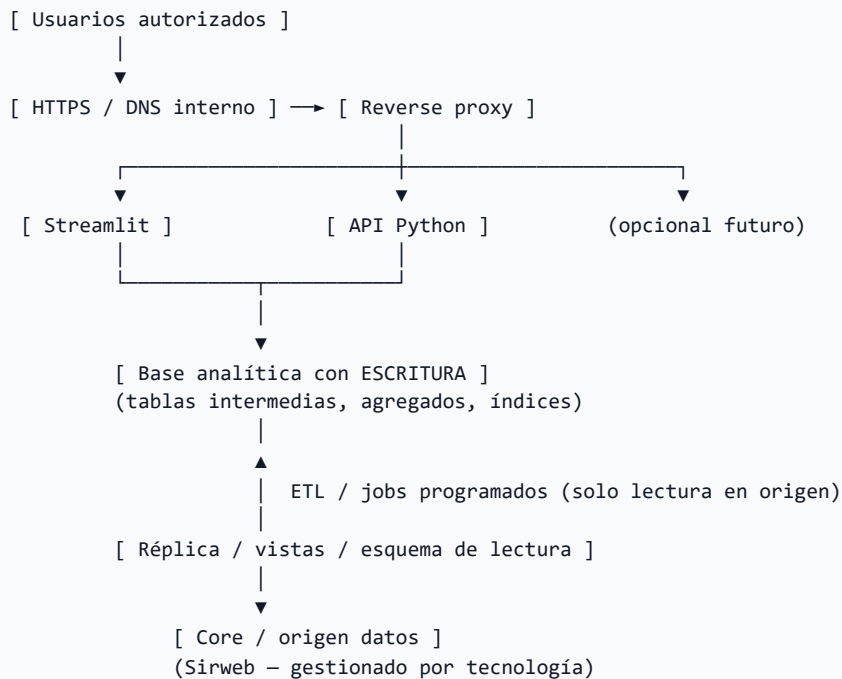
2. Principios de arquitectura

Principio	Descripción
Separación lectura/escritura del core	Las consultas de BI no deben ejecutarse contra el OLTP de producción en horarios o modos que afecten operación.
Solo lectura hacia el origen corporativo	Acceso a datos maestros vía réplica, vistas o esquema analítico provisto por sistemas.
Escritura solo en capa analítica propia	Tablas intermedias, agregados y caches viven en un destino con escritura bajo control del proyecto BI (sección 6).
Contenedores para aplicación	Despliegue con Docker (Compose u homólogo) sobre Ubuntu Server LTS .
Secretos y configuración	Variables por entorno; sin credenciales en repositorio; integración con estándar de la empresa.
Evolución por fases	La arquitectura debe permitir ampliar el BI a otras áreas y fuentes de datos de forma escalonada , con gobierno del dato y capacidad acordadas con tecnología.

3. Alcance funcional (fase inicial)

- **Contexto de cartera:** la oferta comercial actual de La Fe se centra en **funerario, vida AP y RCV**; los indicadores de cobranza deben alinearse a esas líneas.
- **Área y enfoque inicial: seguimiento de la cobranza** (indicadores a definir con negocio: mora, vencimientos, cartera, cumplimiento, etc.).
- **Roadmap:** en **etapas posteriores** se incorporará BI para **otras áreas** de manera escalada; la infraestructura que se instancie debe contemplar ese crecimiento (capacidad, red, separación de entornos o esquemas según política).
- **Tipo de uso:** principalmente lectura y visualización; cualquier escritura hacia sistemas corporativos queda **fuera de alcance** salvo acuerdo explícito posterior.

4. Arquitectura lógica objetivo



5. Infraestructura de aplicaciones (servidor de apps)

5.1 Sistema operativo

Ubuntu Server 22.04 LTS (64 bits), actualizado según política de parches de la empresa.

5.2 Hardware orientativo (VM o físico dedicado)

Recurso	Mínimo (piloto)	Recomendado
CPU	4 vCPU	8 vCPU
RAM	16 GB	32 GB
Disco	SSD 200 GB	SSD ≥ 500 GB
Red	1 Gbps, IP fija en VLAN acordada	

Justificación disco: SO, Docker, imágenes, entornos Python, logs, espacio para cachés locales si aplica política; margen de crecimiento.

5.3 Software base en el servidor de apps

Componente	Uso
Docker + Docker Compose (o equivalente aprobado)	Contenedores de Streamlit, API y procesos batch si se empaquetan así.
Python 3.11+	Runtime alineado con imágenes base o venv si algún componente corre fuera de contenedor.
Reverse proxy (p. ej. nginx)	Terminación TLS, enrutamiento a servicios, límites de tamaño de cuerpo.
Certificados TLS	Certificado corporativo / CA interna (no autofirmado en producción sin excepción documentada).

5.4 Instalación de dependencias y actualizaciones

Se requiere **salida controlada a Internet** vía **proxy corporativo**, o bien **registros internos** (mirror PyPI, registry Docker interno tipo Nexus/Artifactory), según política de La Fe. Sin acceso a repositorios, el despliegue debe apoyarse en **procedimiento documentado** de importación de imágenes y paquetes (air-gap).

5.5 Operación

- Servicios con **arranque automático** y **reinicio** ante fallo.
- **Logs** con rotación; preferible integración con centralización de logs si existe.
- **Backups**: al menos configuración de despliegue y definiciones versionadas en Git; política de backup de volúmenes Docker acordada con sistemas.

5.6 Monitoreo y alertas

Los **logs** y su rotación no bastan para enterarse a tiempo si algo falla: se requiere **visibilidad proactiva** antes de que el fallo se note al **abrir el tablero**. Se pide disponer de **comprobaciones de salud** (*health checks*) sobre al menos la API (Python), el front (Streamlit) y, si aplica, la correcta ejecución de los jobs ETL (p. ej. última corrida, duración anómala o códigos de error).

Cuando un servicio deja de responder o un job crítico no se ejecutó, el entorno debe poder **notificar** al **equipo de desarrollo BI** (y a sistemas si la política lo exige). Como mínimo suele bastar **correo electrónico** hacia una **lista de destinatarios definida** (p. ej. casillas del equipo y, si aplica, de guardia). La herramienta concreta (monitor, agente, integración) la define tecnología en línea con seguridad y coste.

5.7 Continuidad operativa ante incidentes (RTO y RPO)

La política de **backups** debe ir acompañada de **objetivos negociados** con negocio y tecnología sobre cuánto puede estar indisponible el BI y cuánta pérdida de datos es aceptable ante un incidente grave:

- **RTO** (*Recovery Time Objective*, tiempo de recuperación objetivo): tiempo máximo aceptable para volver a tener operativos el tablero y la API tras una caída del servidor u otro evento; importa para la toma de decisiones en cobranza y ayuda a decidir si basta una VM o hace falta **alta disponibilidad**.
- **RPO** (*Recovery Point Objective*, punto de recuperación objetivo): antigüedad máxima aceptable de los datos recuperados; suele alinearse con la frecuencia de respaldo de la capa analítica y volúmenes.

Los valores concretos de RTO y RPO se fijan con el área de tecnología y se revisan al crecer el **uso** y la criticidad.

6. Base de datos y datos

6.1 Lectura desde el entorno corporativo

- **Réplica de solo lectura, vistas** publicadas por sistemas, o **esquema analítico** alimentado por ellos.
- Credenciales **dedicadas** al proyecto BI, **mínimo privilegio** (idealmente solo SELECT sobre objetos acordados).
- Conectividad desde el servidor de apps (o desde el host de ETL) según reglas de firewall.

6.2 Escritura para tablas intermedias

La réplica **no** admite creación de tablas propias del proyecto. **Opción preferida:** instancia **PostgreSQL** (u homólogo aprobado) con **escritura** en esquema dedicado a BI (idealmente **servidor separado** del servidor de aplicaciones). **Opción alternativa (piloto):** DuckDB / SQLite / Parquet en volumen persistente Docker, con acuerdo explícito sobre backup, tamaño y concurrencia.

Ante el **volumen histórico potencial** de datos de cobranza, el paso desde archivos locales o DuckDB/Parquet a un **PostgreSQL dedicado** debe basarse en **criterios de salida del piloto** acordados entre negocio, desarrollo BI y tecnología. A modo orientativo (los umbrales **X** e **Y** se definen en el proyecto):

- Cuando el almacenamiento o la capa analítica supere un tamaño de referencia (p. ej. **X GB**) o deje de cumplirse el rendimiento acordado.
- Cuando la **concurrencia** (accesos simultáneos al tablero y a la API) supere un umbral de referencia (p. ej. más de **Y**) o haya picos sostenidos que degraden los tiempos de respuesta.
- Cuando se requieran capacidades que el esquema de archivos locales no cubre bien (p. ej. integridad concurrente, auditoría, respaldos en caliente según política).

6.3 Separación app vs datos

Escenario	Cuándo
Mismo host: app Docker + motor analítico en contenedor	Piloto pequeño, baja concurrencia; revisar al crecer datos o la carga de accesos.
Hosts separados: apps en un servidor, BD analítica en otro	Escenario ideal para producción y crecimiento.

7. Aplicación (stack de desarrollo)

Capa	Tecnología propuesta
Interfaz BI	Streamlit
API de métricas / negocio	FastAPI (Python)
Validación y contratos	Pydantic / paquetes compartidos en repositorio
Orquestación ETL ligero	Cron, systemd timers, o herramienta que apruebe sistemas

Evolución futura: otro front (p. ej. React) puede consumir la **misma API** sin cambiar el modelo de datos.

Streamlit como capa de presentación: se propone para construir el tablero con rapidez en Python. El **framework Streamlit no incluye de serie** un modelo multiusuario corporativo (cuentas, roles o perfiles de aplicación); **implementarlo sería desarrollo adicional** sobre el marco. En esta fase el **acceso** a la URL del tablero se **restringe por red corporativa, proxy y políticas acordadas** (sección 8), sin asumir ese módulo dentro de la app.

8. Desarrollo, repositorio e integración continua

Usar **Git** (p. ej. vía GitHub u otra forja aprobada) aporta **control de versiones** (historial de cambios, ramas, revisiones), **trabajo colaborativo** y posibilidad de **integración y despliegue automatizados**. No sustituye el almacén de datos de la empresa: solo concentra el **código** y la definición técnica del despliegue.

El trabajo diario se realiza en **estaciones de desarrollo** (p. ej. equipo de escritorio o portátil con capacidad equivalente; ver sección 10). El código se centraliza en un **repositorio privado** en la **forja Git autorizada por la empresa** (creado y mantenido por **desarrollo BI**, con **accesos y política** acordados con tecnología): API (FastAPI), tableros (Streamlit), contenedores y pruebas.

8.1 Qué sí y qué no debe vivir en el repositorio

En el repo **no** deben residir **datos sensibles de negocio** (cartera, pólizas, datos personales), **volcados de bases de datos**, ni **credenciales** en claro (contraseñas, tokens, claves API). Esos elementos permanecen en los **servidores y bases autorizados** o en **gestores de secretos** (sección 9). El repositorio guarda únicamente lo necesario para construir y ejecutar la aplicación: código, plantillas de configuración *sin* secretos, Dockerfile, compose y documentación. Así se reduce el riesgo aunque la forja esté en la nube, siempre en línea con la política de seguridad y cumplimiento de La Fe.

8.2 Contenido mínimo esperado en el repositorio

- **Dockerfile** (uno por servicio o los que aplique): instrucciones para construir la imagen de la aplicación.
- **docker-compose.yml** (u homólogo): orquestación de contenedores en local o integración (app, API, motor analítico del piloto, etc.).
- **.gitignore**: reglas para no versionar secretos, credenciales, entornos locales ni artefactos generados.
- Código fuente (.py), manifiesto de dependencias (requirements.txt o equivalente) y documentación breve de despliegue.

8.3 Despliegue automatizado (p. ej. GitHub Actions)

Una forma eficiente de conectar el repositorio con los **servidores Ubuntu** es usar **GitHub Actions** (o pipeline equivalente en la forja elegida) para ejecutar, ante cambios en la rama acordada, verificaciones, construcción de imágenes y despliegue a integración o producción **según política de La Fe**. Si no se autoriza ejecución desde la nube hacia la red interna, se adoptará la variante aprobada (agente *self-hosted*, publicación de artefactos y despliegue manual supervisado, etc.).

9. Red y seguridad

Tema	Requerimiento
Exposición	Sin publicación abierta a Internet para datos reales; acceso intranet y/o VPN según política.
Autenticación	Fase inicial / piloto: suele bastar que el acceso al tablero esté acotado por red (intranet, VPN, IP permitidas) y políticas que defina tecnología, sin obligar desde el día uno a un SSO integrado al producto. Fases posteriores: integración con directorio / SSO (Azure AD, LDAP, etc.) o proxy con autenticación, cuando negocio y seguridad lo exijan.
Firewall	Reglas explícitas: tráfico desde redes o equipos autorizados → proxy → apps; salida apps → BD lectura y BD analítica.
Cuentas de servicio	Usuario de ejecución sin privilegios de administrador en el SO.
Secretos	Vault, secretos del orquestador, o mecanismo que use La Fe.

10. Entornos

Entorno	Propósito
Desarrollo	Estación de trabajo del desarrollador: equipo asignado (PC Core i7, 16 GB RAM) u ordenador portátil equivalente (p. ej. Intel Core i9-13900H, 16 GB RAM) para el mismo fin.
Integración / pruebas	Servidor o VM con datos anonimizados o subset cuando sea posible.
Producción	Servidor(es) descritos en este documento, datos reales, cambios controlados.

11. Roles y responsabilidades

Área	Responsabilidad
Tecnología / sistemas	VMs, red, firewall, TLS, réplica o acceso lectura, BD analítica con escritura, identidad, backups, monitoreo y alertas acordados, política de salida a Internet o mirrors.
Desarrollo BI	Diseño de aplicación, API, ETL hacia capa analítica, repositorio, documentación de despliegue, no hardcodear secretos.
Negocio / cobranza	Definición de KPIs, validación funcional, uso autorizado.

12. Entregables esperados del área de tecnología (checklist)

Entregable
VM(s) o servidor(es) con especificaciones de la sección 5.2, Ubuntu 22.04 LTS .
Docker y mecanismo de despliegue acordado.
Proxy inverso + HTTPS con certificado válido en el entorno corporativo.
Acceso lectura a datos de cobranza vía réplica/vistas/esquema (credenciales y firewall).
Base de datos con escritura para esquema BI (preferible servidor dedicado) o aprobación explícita de alternativa DuckDB/Parquet con política de backup.
Política de salida a Internet (proxy) o registros internos (PyPI / Docker registry).
(Fase posterior, según política) Integración de autenticación al tablero con directorio / SSO — no requisito bloqueante para un piloto si ya existe control por red/VPN (definir con tecnología y seguridad).
DNS interno para el servicio (nombre estable).
Monitoreo y alertas: health checks (API, Streamlit, jobs ETL según diseño) y notificación proactiva (p. ej. correo a destinatarios acordados).
RTO y RPO documentados y alineados con negocio (continuidad del tablero ante incidentes).
Forja Git autorizada por la empresa con repositorio privado del proyecto (creado y mantenido por desarrollo BI) ; el área de tecnología facilita accesos y política). Lo que corre en el servidor de aplicaciones debe ser la réplica completa del desarrollo desplegado; el repositorio no sustituye al servidor: es el medio para incorporar cambios con agilidad , versionar el código y permitir que el desarrollador publique mejoras en cualquier momento (según ramas y política acordadas). Incluye Dockerfile, docker-compose (o equivalente), .gitignore y pipeline de integración/despliegue acordado (p. ej. GitHub Actions o alternativa interna).

13. Supuestos y puntos abiertos

- El modelo exacto de datos de cobranza y nombres de tablas/vistas lo define **negocio + tecnología** con el core Sirweb, alineado a las líneas **funerario, vida AP y RCV**.
- La ampliación del BI a **otras áreas** y el calendario por fases lo acuerdan **negocio y tecnología**; la infraestructura descrita debe poder **soportar esa evolución** sin partir de cero en cada etapa.
- El dimensionamiento fino puede ajustarse tras un **piloto de carga** y al incorporar nuevas áreas o mayor uso concurrente.
- Cualquier cambio que implique **escritura en sistemas transaccionales** del core queda **fuera** de este documento hasta nuevo acuerdo.

14. Planificación indicativa (un recurso a tiempo parcial) y fases posteriores

Advertencia: de **carácter meramente orientativo**. Lo que más condiciona el calendario suele ser **infraestructura, datos y aprobaciones**. El alcance del proyecto incluye **desarrollar las consultas** al origen de datos autorizado y los **procesos ETL** hacia la capa analítica (sección 6).

Supuesto: un recurso a tiempo parcial asignado al proyecto; piloto acotado (cobranza; líneas funerario, vida AP y RCV); y colaboración oportuna del área de tecnología en accesos y despliegue. Un orden de magnitud razonable sería:

- ~1,5–2,5 meses: primer entregable en **integración / pruebas** (API + tablero inicial + ETL mínimo).
- ~3–5 meses: **piloto en producción** con valor para el **área de negocio** (despliegue estable, alertas básicas; revisar sección 6.2).
- Si sistemas o seguridad alargan los trámites, el calendario se desplaza **sin relación** con la dedicación del recurso.

Definir por escrito un **MVP** del piloto (pantallas e indicadores) y la fecha objetivo de disponibilidad de datos ayuda a acotar estos plazos.

14.1 Fases posteriores (solo esquema; sin fechas)

- **Fase posterior 1 — Consolidación de cobranza:** más KPIs y mayor uso del tablero, monitoreo y backups acordados, salto a motor analítico dedicado cuando proceda (sección 6.2).
- **Fase posterior 2 — Otras áreas de BI:** nuevos tableros y fuentes sobre la misma arquitectura, con permisos y gobierno del dato por ámbito.
- **Fase posterior 3 — Madurez:** autenticación corporativa si aplica, capacidad y continuidad alineadas a criticidad y RTO/RPO.

14.2 Seguimiento de avances (orientativo)

Conviene acordar una **cadencia de seguimiento** entre desarrollo BI, tecnología y, cuando aplique, negocio; lo siguiente es **meramente orientativo** y no sustituye el calendario de reuniones que defina la empresa:

- **Fase de arranque** (infraestructura y acceso a datos): **sincronización semanal breve** (p. ej. 15–30 minutos) o **informe escrito** de hecho / próximo paso / bloqueos.
- **Revisión con lista de logros o demo corta** aproximadamente cada **dos semanas**, como equilibrio entre visibilidad y carga administrativa.
- **Reunión o informe mensual** con dirección o negocio puede reservarse para **alcance y prioridades**, no para el detalle técnico cotidiano.
- Si no hay novedades relevantes, la sesión semanal puede **cancelarse o sustituirse por comunicación asíncrona**.

Anexo A. Glosario — términos en lenguaje sencillo

Si alguna palabra le suena a “jerga”, use esta tabla como ayuda rápida. Cuando una sigla corresponde a un nombre en inglés, se indica entre paréntesis o en cursiva, sin recargar el texto. Del negocio a la tecnología que lo hace posible.

Término	Qué significa aquí
BI / Inteligencia de negocio	En inglés se usa mucho <i>Business Intelligence</i> ; aquí significa usar datos para medir y entender el negocio (indicadores, tableros, tendencias), no para reemplazar el sistema operativo de pólizas.

Término	Qué significa aquí
KPI	Siglas de <i>Key Performance Indicator</i> —indicador clave de rendimiento—. Por ejemplo: mora, vencimientos o montos según la cartera (funerario, vida AP, RCV). Se define con negocio y se calcula a partir de los datos autorizados.
Streamlit	Herramienta en Python para construir pantallas web de datos (gráficos, filtros) con rapidez. El marco no trae de fábrica gestión de cuentas de aplicación tipo producto multiusuario; añadirla implica desarrollo adicional . El acceso se combina con red, proxy y políticas (sección 8). En la fase inicial cubre el tablero de cobranza y, más adelante, otras áreas según el roadmap.
API (interfaz de programación)	Corresponde a <i>Application Programming Interface</i> (interfaz de programación de aplicaciones): un servicio que “sirve” números y reglas por la red interna; la pantalla pregunta y la API responde de forma ordenada. Aquí se propone FastAPI (framework de API en Python, nombre del proyecto).
Python	Lenguaje de programación en el que se desarrollarán las aplicaciones de análisis y la API; facilita análisis posteriores y integración con librerías de datos.
Docker / contenedor	<i>Docker</i> es el nombre de la plataforma; en inglés <i>container</i> es el “contenedor” donde va empaquetada la app. Forma de empaquetar cada aplicación con todo lo que necesita para correr, igual en pruebas y en servidor, sin instalar “a mano” cada librería en el sistema operativo.
Ubuntu Server	Sistema operativo Linux orientado a servidores. La marca LTS viene de <i>Long-Term Support</i> : versión con soporte prolongado. Versión pedida: 22.04 LTS .
Core / Sirweb	<i>Core</i> , en jerga de sistemas, es el núcleo o sistema central. Sirweb es el sistema transaccional donde se registra la operación cotidiana. No es el lugar donde conviene ejecutar reportes pesados de BI.
OLTP	Siglas de <i>Online Transaction Processing</i> (procesamiento de transacciones en línea): tipo de uso de base de datos pensado para muchas operaciones del día a día (pagos, estados de cuenta). Debe mantenerse ágil.
Réplica (solo lectura)	Copia de los datos actualizada para consultar , sin mezclar con la escritura del core (en inglés suele decirse <i>read replica</i> o copia de solo lectura). Ideal para alimentar reportes sin tocar producción.
Base analítica (con escritura)	Otro motor de base de datos (o archivos controlados) donde el proyecto BI puede crear tablas propias de resumen; no sustituye al core.
ETL	Siglas de <i>Extract, Transform, Load</i> —extraer, transformar y cargar datos—: procesos que extraen datos del origen, los transforman y los cargan donde corresponda (por ejemplo, hacia la base analítica).
HTTPS / TLS	HTTPS es HTTP con cifrado (<i>Hypertext Transfer Protocol Secure</i>); TLS es la capa criptográfica (<i>Transport Layer Security</i> , seguridad de la capa de transporte). Navegación cifrada; evita que datos sensibles circulen en claro por la red interna.
Proxy inverso	En inglés <i>reverse proxy</i> : componente delante de las aplicaciones que centraliza el acceso por nombre web, certificados y reglas de entrada.

Término	Qué significa aquí
DNS interno	DNS son las siglas de <i>Domain Name System</i> (sistema de nombres de dominio): convierte nombres en direcciones. Aquí se refiere a un nombre estable para el servicio (por ejemplo <i>bi-cobranza.empresa</i>) en lugar de una dirección numérica difícil de recordar.
VPN / intranet	VPN viene de <i>Virtual Private Network</i> (red privada virtual). La intranet es la red interna de la organización. Forma de acceder a sistemas internos solo desde redes o equipos autorizados, según política de La Fe.
SSO / directorio (LDAP, Azure AD, etc.)	SSO es <i>Single Sign-On</i> (inicio de sesión único). LDAP son las siglas de <i>Lightweight Directory Access Protocol</i> (acceso ligero a directorios). "Azure AD" alude al directorio de Microsoft (hoy a menudo bajo la marca Entra ID). Inicio de sesión unificado con el usuario corporativo, en lugar de contraseñas sueltas por aplicación.
Firewall	Término inglés que en español se traduce por <i>cortafuegos</i> : reglas de red que permiten solo los caminos necesarios (por ejemplo, de la app hacia las bases de datos autorizadas).
VM (máquina virtual)	Siglas de <i>Virtual Machine</i> . "Computadora simulada" dentro del hardware de la empresa; permite asignar CPU, RAM y disco sin comprar un equipo físico nuevo en cada caso.
SSD	Siglas de <i>Solid-State Drive</i> (unidad de estado sólido). Disco rápido; recomendable para que consultas y aplicaciones respondan bien.
PostgreSQL	Base de datos relacional de uso muy extendido (nombre del proyecto de software libre); candidata habitual para la capa analítica con escritura .
PyPI / registry Docker	PyPI es el <i>Python Package Index</i> (índice de paquetes de Python). Un <i>registry</i> de Docker es el registro donde se publican las imágenes de contenedor. Repositorios públicos de descarga; en empresas a menudo se usan copias internas o proxy por seguridad.
RTO / RPO	RTO (<i>Recovery Time Objective</i>): tiempo máximo aceptable para restaurar el servicio tras una caída. RPO (<i>Recovery Point Objective</i>): cuán "viejos" pueden ser los datos recuperados. Sirven para planificar backups y alta disponibilidad.
Health check	Comprobación automática de que un servicio "está vivo" (p. ej. la API responde, el front carga, un job terminó bien). Complementa los logs para detectar fallos antes de que el problema se manifieste en el tablero.
GitHub Actions	Servicio de <i>integración continua</i> en GitHub: ejecuta flujos (pruebas, construcción, despliegue) al hacer <i>push</i> o según cron; debe ajustarse a la política de red y seguridad de La Fe.